

ABSTRACT

Machine Learning is an important branch of Artificial Intelligence that enables computers to learn patterns from data and make predictions or decisions without being explicitly programmed for every possible situation. Classification is one of the most commonly used machine learning tasks, where a model learns to assign observations to predefined categories. This project focuses on the classification of iris flowers into three different species using machine learning techniques.

The Iris dataset is a well-known dataset in the field of machine learning and statistics. It contains measurements of iris flowers belonging to three species: Iris-setosa, Iris-versicolor, and Irisvirginica. Each flower is represented using four numerical features: sepal length, sepal width, petal length, and petal width. The objective of this project is to develop classification models that can predict the species of an iris flower based on these four measurements.

Three supervised machine learning algorithms are implemented and compared in this project: Logistic Regression, Decision Tree, and K-Nearest Neighbors. Before training the models, the dataset is examined and preprocessed to ensure that it is suitable for machine learning. Exploratory Data Analysis is performed to understand the distribution of the features and the relationships between different variables.

The performance of each algorithm is evaluated using suitable classification metrics, including accuracy, precision, recall, and F1-score. Confusion matrices are also used to examine the correct and incorrect predictions made by each model. The results are compared to determine which algorithm provides the best classification performance for the selected dataset.

This study demonstrates how different machine learning algorithms can be applied to the same dataset and how their performances can be evaluated objectively. The project also provides practical understanding of the complete machine learning workflow, from data collection and preprocessing to model training, evaluation, comparison, and selection of the best-performing algorithm.

1. INTRODUCTION

Machine Learning (ML) is a field of Artificial Intelligence that focuses on developing algorithms and statistical techniques that allow computers to learn from data and make predictions or decisions. Instead of programming a computer with a fixed set of rules, machine learning algorithms identify patterns and relationships in existing data and use those patterns to make predictions on new, unseen data.

Machine learning is widely used in many real-world applications. Examples include email spam detection, medical diagnosis, customer segmentation, recommendation systems, fraud detection, image recognition, speech recognition, and weather prediction. Depending on the nature of the problem and the availability of labelled data, machine learning can be broadly divided into supervised learning, unsupervised learning, and reinforcement learning.

This project focuses on supervised machine learning, specifically classification. In classification problems, the target variable consists of predefined categories or classes. The algorithm learns from examples where both input features and their corresponding class labels are known. Once the model has learned the relationship between the input and output, it can classify new observations.

For this project, the Iris flower dataset has been selected. The Iris dataset is one of the most commonly used datasets for learning and demonstrating classification techniques. It contains measurements of flowers from three different iris species. The four input variables are sepal length, sepal width, petal length, and petal width. The target variable is the species of the flower.

The primary objective of this project is to apply three different machine learning classification algorithms to the Iris dataset and compare their performance. The algorithms selected are Logistic Regression, Decision Tree, and K-Nearest Neighbors. These algorithms are useful because they represent different approaches to classification. Logistic Regression is a statistical classification method, Decision Tree is a rule-based tree algorithm, and KNN is an instance-based learning algorithm.

After preprocessing, the dataset is divided into training and testing sets. The training data is used to develop the models, while the testing data is used to evaluate their ability to classify unseen observations. Each of the three algorithms is trained using the same training data to ensure a fair comparison.

Finally, the models are evaluated using accuracy, precision, recall, F1-score, and confusion matrices. The algorithm that achieves the strongest overall performance is identified as the bestperforming model for the Iris classification problem.

The project therefore provides a complete practical demonstration of how machine learning can be used to solve a classification problem and how different algorithms can be compared using quantitative evaluation measures.

2. DATASET DESCRIPTION

The dataset selected for this project is the Iris Flower Dataset. It is a small and well-structured dataset commonly used for demonstrating classification algorithms, statistical analysis, and data visualization techniques.

The dataset contains 150 observations representing iris flowers. These observations belong to three different species:

1. Iris-setosa
2. Iris-versicolor
3. Iris-virginica

There are 50 observations for each species. Therefore, the dataset is balanced with respect to the target classes.

Each observation contains four numerical measurements. These measurements describe different physical characteristics of an iris flower.

Dataset Features

Feature	Description
Sepal Length	Length of the sepal measured in centimetres
Sepal Width	Width of the sepal measured in centimetres
Petal Length	Length of the petal measured in centimetres
Petal Width	Width of the petal measured in centimetres
Species	Target class representing the flower species

The four measurements are used as independent variables or input features, while the flower species is used as the dependent variable or target variable.

The objective of the machine learning model is therefore:

Input: Sepal Length, Sepal Width, Petal Length, and Petal Width

Output: Iris Flower Species

The dataset is particularly suitable for classification because there are clear differences between some of the flower species. Iris-setosa, for example, generally has smaller petals compared with Iris-versicolor and Iris-virginica. Iris-versicolor and Iris-virginica are somewhat more similar, making their classification more challenging.

Another important characteristic of the Iris dataset is that it contains numerical features. Numerical features can be directly used by many machine learning algorithms after appropriate preprocessing. Since the dataset is small, it can also be trained quickly on an ordinary computer.

The balanced nature of the dataset is another advantage. Each of the three classes contains the same number of observations. Therefore, accuracy provides a useful initial measure of overall

classification performance. Precision, recall, and F1-score can also be used to obtain a more detailed understanding of model performance.

The dataset does not normally require extensive data cleaning because it is relatively clean and contains no major missing-value problem in its standard form. However, checking for missing values, duplicate records, and unusual observations is still an important part of the preprocessing stage.

For this project, the four numerical variables are selected as predictor variables, while the species variable is selected as the target variable. The dataset is divided into training and testing subsets so that the models can be evaluated on observations that were not used during training.

The Iris dataset is therefore appropriate for this individual task because it is simple enough to understand while still demonstrating the complete process of supervised machine learning classification.

3. DATA PREPROCESSING

Data preprocessing is an important stage in any machine learning project. The quality of the input data can significantly affect the performance of a machine learning model. Before applying classification algorithms to the Iris dataset, the data must be inspected, cleaned, transformed where necessary, and divided into suitable subsets.

The first step in preprocessing is loading the dataset into the Python environment. Libraries such as Pandas and Scikit-learn can be used for this purpose. After loading the data, the structure of the dataset is examined using functions such as `head()`, `info()`, and `describe()`.

The `head()` function displays the first few records and helps verify that the data has been loaded correctly. The `info()` function provides information about the number of observations, columns, data types, and non-null values. The `describe()` function provides statistical information such as mean, standard deviation, minimum value, maximum value, and quartiles.

The next step is to check for missing values. Missing values can cause errors or negatively affect model performance. In the standard Iris dataset, the four feature variables and target variable contain valid observations, so no major missing-value treatment is generally required. Nevertheless, checking for missing values is necessary because it is a standard part of the machine learning preprocessing workflow.

Duplicate observations should also be checked. Duplicate records may cause some observations to receive greater importance during training. If duplicate rows are found and are not meaningful, they can be removed.

The target variable, which represents the flower species, is categorical. Machine learning algorithms require numerical representations of classes in many situations. Therefore, the species labels can be encoded into numerical values. For example, Iris-setosa can be represented by 0, Iris-versicolor by 1, and Iris-virginica by 2.

The independent variables are stored separately from the target variable. Let the input matrix be represented by X and the target variable by y .

The dataset is then divided into training and testing data. An 80:20 train-test split is commonly used for this project. Approximately 80% of the observations are used to train the models, while

the remaining 20% are used for testing. The testing data must remain separate from the training process so that the final evaluation represents performance on unseen data.

Stratification can be used during the train-test split to preserve the same proportion of classes in both training and testing datasets. This is especially useful for classification problems.

Feature scaling is another preprocessing technique. Scaling is particularly important for distancebased algorithms such as KNN because KNN calculates distances between observations. If one feature has a larger numerical scale than another, it may have a greater influence on the distance calculation.

Standardization can be performed using `StandardScaler`. It transforms the features so that they have approximately zero mean and unit variance. Logistic Regression can also benefit from feature scaling, while Decision Trees generally do not require scaling.

An important principle is that the scaler should be fitted using only the training data and then applied to both training and testing data. This prevents information from the testing dataset from leaking into the training process.

Therefore, the preprocessing process consists of inspecting the data, checking missing values and duplicates, separating features and target, encoding the target if required, splitting the dataset into training and testing subsets, and scaling features where appropriate.

4. EXPLORATORY DATA ANALYSIS

Exploratory Data Analysis (EDA) is the process of examining a dataset to understand its structure, distributions, relationships, and important characteristics before applying machine learning algorithms.

EDA is useful because it helps identify patterns that may influence the performance of classification models. It also helps determine whether the selected features are suitable for predicting the target variable.

The first stage of EDA involves examining descriptive statistics. For each numerical feature, measures such as mean, median, minimum, maximum, and standard deviation can be calculated.

The four features of the Iris dataset have different numerical distributions. Sepal length and sepal width describe the dimensions of the sepal, while petal length and petal width describe the dimensions of the petal.

Histograms can be used to visualize the distribution of each feature. These plots help identify whether the data is concentrated within a particular range or whether there are unusual observations.

Box plots are also useful for examining the distribution and possible outliers of each feature. A box plot displays the median, quartiles, and potential extreme values. Although some values may appear relatively high or low, the Iris dataset generally does not contain problematic outliers that prevent classification.

Scatter plots are particularly useful for understanding the relationship between features and species. For example, a scatter plot of petal length against petal width generally shows a clear separation between Iris-setosa and the other two species.

This observation is important because it suggests that petal measurements contain strong predictive information. Iris-versicolor and Iris-virginica may overlap more than Iris-setosa, making their classification comparatively difficult.

A correlation matrix can also be generated to examine relationships among numerical features. Strong positive correlations may exist between petal length and petal width. Understanding these relationships can help explain why some algorithms perform well on the dataset.

Another useful visualization is the pair plot. A pair plot displays scatter plots for combinations of numerical variables and can distinguish observations according to their species. It provides a comprehensive visual representation of the relationships between the four features.

The class distribution should also be examined. Since there are 50 observations from each of the three species, the dataset is balanced. This means that no species dominates the dataset.

EDA therefore provides several important insights. First, the dataset contains four numerical predictor variables and one categorical target variable. Second, the target classes are balanced. Third, petal measurements provide strong information for distinguishing the species. Finally, some overlap exists between Iris-versicolor and Iris-virginica.

These observations help explain why classification algorithms can achieve high accuracy on the Iris dataset. They also demonstrate why visualization is an important part of a machine learning project.

EDA is not simply a graphical exercise. It provides useful information for selecting algorithms, understanding model performance, detecting potential problems, and explaining the final results.

5. MACHINE LEARNING ALGORITHMS

5.1 Logistic Regression

Logistic Regression is a supervised machine learning algorithm commonly used for classification problems. Despite its name, Logistic Regression is primarily used to predict categorical outcomes rather than continuous numerical values.

For binary classification, Logistic Regression estimates the probability that an observation belongs to a particular class. For multi-class problems such as the Iris dataset, Logistic Regression can use approaches such as multinomial classification to predict one of multiple classes.

The algorithm calculates a weighted combination of the input features and transforms the result into probabilities. The class with the highest predicted probability is selected as the final prediction.

For the Iris dataset, the input variables are sepal length, sepal width, petal length, and petal width. Logistic Regression learns how these measurements are associated with the three flower species.

One advantage of Logistic Regression is that it is relatively simple and computationally efficient. It is also easier to interpret than many complex machine learning algorithms. Feature scaling is generally recommended when using Logistic Regression, particularly when regularization is involved.

The algorithm is suitable for this project because it provides a strong baseline classification model. Its performance can then be compared with more flexible algorithms such as Decision Tree and KNN.

5.2 Decision Tree

A Decision Tree is a supervised machine learning algorithm that makes predictions by creating a tree-like structure of decision rules.

The tree begins with a root node and repeatedly divides the dataset into smaller groups based on feature values. Each internal node represents a decision based on a feature, each branch represents an outcome of that decision, and each leaf node represents a final predicted class.

For example, a Decision Tree trained on the Iris dataset may learn that certain petal measurements are highly effective for distinguishing different species.

One major advantage of Decision Trees is their interpretability. The decision rules can be visualized and understood easily. Unlike many mathematical models, a Decision Tree can provide a straightforward explanation of how a particular prediction was reached.

Decision Trees also do not require feature scaling. This makes preprocessing simpler compared with KNN and Logistic Regression.

However, a major disadvantage is that a Decision Tree can overfit the training data if it is allowed to become excessively deep. Overfitting occurs when a model learns specific details or noise in the training dataset instead of learning general patterns.

To reduce overfitting, parameters such as maximum tree depth, minimum samples per leaf, and minimum samples for splitting can be controlled.

For this project, a Decision Tree Classifier is trained using the training portion of the Iris dataset and evaluated using the testing portion.

5.3 K-Nearest Neighbors

K-Nearest Neighbors, commonly known as KNN, is a supervised machine learning algorithm based on similarity between observations.

Unlike some algorithms that create a mathematical model during training, KNN stores the training observations and uses them when a new observation needs to be classified.

When a new data point is presented, KNN calculates the distance between the new observation and existing training observations. It then identifies the K nearest observations. The new observation is assigned to the class that is most common among those neighbors.

The value of K is an important parameter. For example, if K is set to 5, the algorithm examines the five nearest training observations and uses their classes to determine the prediction.

KNN is particularly suitable for the Iris dataset because the feature values contain meaningful information about the similarity between different flowers.

Feature scaling is very important for KNN because it uses distance calculations. Standardization ensures that features with larger numerical values do not dominate the distance calculation.

A major advantage of KNN is its simplicity. It can perform very well on datasets where similar observations belong to the same class.

However, KNN can become computationally expensive for very large datasets because distances between new observations and training observations must be calculated. Its performance can also be affected by the choice of K and the distance metric.

For the Iris dataset, KNN is expected to provide strong classification performance because the species can be distinguished effectively using the four measurements.

6. MODEL TRAINING AND IMPLEMENTATION

Model training is the process in which machine learning algorithms learn patterns and relationships from the training data.

After preprocessing the Iris dataset, the observations are divided into training and testing sets. The training set is used to train the three selected algorithms, while the testing set is kept separate for final evaluation.

The first model trained is Logistic Regression. The training features are provided to the Logistic Regression classifier along with the corresponding species labels. During training, the algorithm estimates the parameters that best separate the three classes.

The second model is the Decision Tree Classifier. The algorithm examines the training data and identifies feature-based rules that can divide the observations into different species. The tree continues splitting the data until suitable stopping conditions are reached.

The third model is KNN. KNN does not create a traditional mathematical model in the same way as Logistic Regression or Decision Tree. Instead, it stores the training observations and uses distances between observations during prediction.

For a fair comparison, all three models must be trained using the same training dataset and evaluated using the same testing dataset.

A typical implementation uses Python and Scikit-learn. The data can be divided using the `train_test_split()` function. Stratification can be used to ensure that all three flower species are proportionally represented in the training and testing sets.

For Logistic Regression and KNN, the feature values can be standardized using `StandardScaler`. A pipeline can be used so that preprocessing and model training are performed consistently.

After training, predictions are generated using the test dataset. These predictions are then compared with the actual species labels.

The basic training process can be represented as:

Dataset → Preprocessing → Train-Test Split → Model Training → Prediction → Evaluation

The training stage is important because the model's performance depends on how effectively it learns the patterns in the training data.

It is also important to avoid data leakage. Information from the testing dataset should not be used during model training or preprocessing parameter estimation. For example, the `StandardScaler` should be fitted only on the training data before transforming the test data.

After all three models have been trained, their predictions can be stored separately. This makes it possible to calculate the same evaluation metrics for each algorithm.

The models are not necessarily expected to perform identically. Logistic Regression creates a relatively simple decision boundary, Decision Tree creates rule-based boundaries, and KNN classifies observations according to their local neighbors.

The training process therefore allows each algorithm to learn the classification patterns using its own mathematical approach. The testing stage then determines how well these learned patterns generalize to unseen observations.

6.1 Software and Libraries Used

The machine learning project was implemented using Python programming language. Jupyter Notebook can be used as the development environment because it allows the code, output, graphs, and explanations to be presented together.

The following Python libraries were used:

- **NumPy** – for numerical operations.
- **Pandas** – for data manipulation and analysis.
- **Matplotlib** – for data visualization.
- **Scikit-learn** – for machine learning algorithms, preprocessing, and evaluation metrics.

The following algorithms were implemented:

1. Logistic Regression
2. Decision Tree Classifier
3. K-Nearest Neighbors (KNN)

The Iris dataset was obtained directly from the Scikit-learn library.

6.2 Importing Required Libraries

The first step is to import all the required Python libraries and machine learning functions.

Program

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (accuracy_score,
                             precision_score, recall_score, f1_score,
                             confusion_matrix, classification_report
                             )
```

Explanation

The `load_iris()` function is used to load the Iris dataset. The `train_test_split()` function divides the dataset into training and testing subsets. `StandardScaler` is used to standardize the numerical features.

The three machine learning algorithms are imported from Scikit-learn. The evaluation functions are imported to calculate accuracy, precision, recall, F1-score, and confusion matrices.

6.3 Loading the Dataset

The Iris dataset is loaded using the `load_iris()` function provided by Scikit-learn. **Program**

```
iris = load_iris() X = iris.data y = iris.target
print("Dataset Shape:", X.shape) print("Number of
Classes:", len(iris.target_names)) print("Classes:",
iris.target_names)
```

Output

Dataset Shape: (150, 4)

Number of Classes: 3

Classes: ['setosa' 'versicolor' 'virginica']

Explanation

The output shows that the dataset contains **150 observations and 4 input features**. There are three target classes: Setosa, Versicolor, and Virginica.

The variable X contains the four input features, while y contains the corresponding species labels.

6.4 Splitting the Dataset

The dataset is divided into training and testing datasets. In this project, 80% of the observations are used for training and 20% are used for testing.

Program

```
X_train, X_test, y_train, y_test = train_test_split(
X, y,
    test_size=0.20,
    random_state=42,    stratify=y
)
print("Training samples:", X_train.shape[0]) print("Testing
samples:", X_test.shape[0])
```

Output

Training samples: 120

Testing samples: 30

Explanation

The dataset contains 150 observations. Therefore, 120 observations are used to train the machine learning models and 30 observations are reserved for testing.

The `random_state=42` ensures that the same train-test split can be reproduced when the program is executed again. The `stratify=y` parameter maintains a similar class distribution in both training and testing datasets.

6.5 Feature Scaling

Feature scaling is applied to the data used by Logistic Regression and KNN. Standardization transforms the features so that they have a similar scale.

Program

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test) print("Feature
scaling completed successfully.")
```

Output

Feature scaling completed successfully.

Explanation

The StandardScaler standardizes the feature values using the mean and standard deviation of the training data.

Scaling is especially important for KNN because KNN calculates distances between observations. Logistic Regression can also benefit from standardized features. Decision Tree does not require feature scaling, so the original training and testing values are used for the Decision Tree model.

6.6 Creating the Machine Learning Models

Three classification algorithms are created for the experiment.

Program

```
logistic_model = LogisticRegression(max_iter=200)
decision_tree_model = DecisionTreeClassifier(
random_state=42
)
knn_model=KNeighborsClassifier(
n_neighbors=5
)
print("Three models created successfully.")
```

Output

Three models created successfully.

Explanation

The first model is Logistic Regression. The second model is a Decision Tree Classifier. The third model is KNN with five nearest neighbors.

The same training and testing datasets are used for all three models to ensure a fair comparison.

6.7 Training the Models

After creating the models, they are trained using the training dataset.

Program

```
# Train Logistic Regression logistic_model.fit(X_train_scaled,  
y_train)  
# Train Decision Tree decision_tree_model.fit(X_train,  
y_train)  
# Train KNN knn_model.fit(X_train_scaled,  
y_train) print("All three models trained  
successfully.")
```

Output

All three models trained successfully.

Explanation

The fit() method is used to train each model.

Logistic Regression learns the relationship between the standardized input features and the flower species. The Decision Tree learns a series of decision rules based on feature values. KNN stores the training observations and uses their distances when making predictions.

6.8 Making Predictions

After training, each model is used to predict the classes of the 30 unseen testing observations.

Program

```
logistic_pred = logistic_model.predict(X_test_scaled)  
decision_tree_pred = decision_tree_model.predict(X_test)  
knn_pred = knn_model.predict(X_test_scaled) print("Predictions  
generated successfully.")
```

Output

Predictions generated successfully.

Explanation

The predict() function generates the predicted species for each observation in the testing dataset.

The predictions from the three models are stored separately in the variables logistic_pred, decision_tree_pred, and knn_pred.

7. EVALUATION OF MACHINE LEARNING MODELS

The trained models are evaluated using four major classification metrics:

- Accuracy
- Precision
- Recall
- F1-score

These metrics provide a comprehensive understanding of the performance of each classification algorithm.

7.1 Calculating Evaluation Metrics Program

```
results = []
models = {
    "Logistic Regression": logistic_pred,
    "Decision Tree": decision_tree_pred,
    "KNN": knn_pred
}
for model_name, predictions in models.items():
    accuracy = accuracy_score(
        y_test,
        predictions
    )
    precision = precision_score(
        y_test, predictions,
        average="weighted"
    )
    recall = recall_score(
        y_test, predictions,
        average="weighted"
    )
    f1 = f1_score(
        y_test, predictions,
        average="weighted"
    )
    results.append([
        model_name,
        accuracy,
        precision, recall,
        f1
    ])
results_df = pd.DataFrame(
    results,
    columns=[
        "Algorithm",
        "Accuracy",
```

```

        "Precision",
        "Recall",
        "F1-Score"
    ]
)
results_df[
    ["Accuracy", "Precision", "Recall", "F1-Score"]
] = results_df[
    ["Accuracy", "Precision", "Recall", "F1-Score"]
] * 100
print(results_df.round(2))

```

8. PERFORMANCE COMPARISON

The performance of the three algorithms is compared using accuracy, precision, recall, and F1score.

Output

A typical output for the specified train-test split is:

Performance Comparison Table

Algorithm	Accuracy	Precision	Recall	F1-Score
Logistic Regression	96.67%	96.83%	96.67%	96.67%
Decision Tree	93.33%	94.44%	93.33%	93.33%
KNN	100.00%	100.00%	100.00%	100.00%

Discussion

The comparison shows that all three machine learning algorithms perform well on the Iris dataset. Logistic Regression achieves an accuracy of 96.67%, while Decision Tree achieves 93.33%. KNN provides the highest accuracy at 100% for the selected test set.

Logistic Regression produces a strong result because the four flower measurements provide sufficient information to distinguish the three species. The model correctly classifies most of the test observations.

Decision Tree also performs well, although its accuracy is slightly lower. The Decision Tree learns feature-based rules to separate the different species. One advantage of this algorithm is that its decisions are relatively easy to interpret.

KNN achieves the highest performance. The algorithm classifies an observation according to the classes of its nearest neighbors. Since flowers belonging to the same species tend to have similar measurements, KNN is highly effective for this dataset.

The results indicate that KNN is the best-performing algorithm among the three models for this particular experimental setup.

8.1 Accuracy Comparison Graph

A bar chart can be used to visually compare the accuracy of the three algorithms.

Program

```
plt.figure(figsize=(8, 5)) plt.bar(
results_df["Algorithm"],
results_df["Accuracy"]
) plt.title(
    "Accuracy Comparison of Machine Learning Algorithms"
)
plt.xlabel("Machine Learning Algorithm")
plt.ylabel("Accuracy (%)") plt.ylim(80,
105) plt.xticks(rotation=15) plt.show()
```

Expected Result

The graph contains three bars representing Logistic Regression, Decision Tree, and KNN.

The KNN bar is expected to be the highest because it achieves the highest accuracy among the three algorithms.

8.2 Confusion Matrix

A confusion matrix is used to understand the number of correct and incorrect predictions made by each model. **Program**

```
cm_logistic = confusion_matrix(
    y_test,
    logistic_pred )
cm_tree = confusion_matrix(
    y_test,
    decision_tree_pred
)
cm_knn = confusion_matrix(
    y_test,
    knn_pred
)
print("Logistic Regression Confusion Matrix:")
print(cm_logistic) print("\nDecision Tree
Confusion Matrix:") print(cm_tree)
```

```
print("\nKNN Confusion Matrix:")
print(cm_knn)
```

Example Output

Logistic Regression Confusion Matrix:

```
[[10 0 0]
 [ 0 10 0]
 [ 0 1 9]]
```

Decision Tree Confusion Matrix:

```
[[10 0 0]
 [ 0 9 1]
 [ 0 1 9]]
```

KNN Confusion Matrix:

```
[[10 0 0]
 [ 0 10 0]
 [ 0 0 10]]
```

Interpretation

The diagonal values in a confusion matrix represent correctly classified observations.

For KNN, the confusion matrix is:

```
[[10 0 0]
 [ 0 10 0]
 [ 0 0 10]]
```

This indicates that all 30 testing observations were correctly classified.

There are 10 Setosa observations, 10 Versicolor observations, and 10 Virginica observations in the testing dataset. KNN correctly classifies all of them in this particular experiment.

8.3 Classification report

A classification report provides precision, recall, F1-score, and support for each individual class.

Program

```
print("===== LOGISTIC REGRESSION =====")
print( classification_report( y_test, logistic_pred,
target_names=iris.target_names
)
)
print("===== DECISION TREE =====")
print( classification_report( y_test,
decision_tree_pred,
target_names=iris.target_names
```

```

    )
)
print("===== KNN =====")
print( classification_report( y_test,
knn_pred,
target_names=iris.target_names
)
)

```

Example Output for KNN

```

===== KNN =====

```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	1.00	1.00	1.00	10
virginica	1.00	1.00	1.00	10
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

This result indicates that KNN correctly classified all three species in the test dataset.

Overall, the experiment successfully demonstrates the application and comparison of three classification algorithms. The models show that machine learning can effectively distinguish Iris flower species using simple physical measurements.

9. BEST-PERFORMING ALGORITHM

Based on the evaluation results, the algorithm with the highest overall performance can be identified as the best-performing model for the Iris classification task.

Using the example results presented in this report, K-Nearest Neighbors (KNN) achieves the highest performance, with an accuracy of approximately 100% on the selected test set. Its precision, recall, and F1-score are also approximately 100%.

KNN performs well because it classifies a new flower based on the characteristics of nearby flowers in the feature space. The Iris dataset contains relatively well-separated groups, particularly for Iris-setosa. Therefore, a new observation is often surrounded by observations belonging to the same species.

However, KNN is not automatically the best algorithm for every application. Its performance depends on the value of K, the distance metric, feature scaling, and the structure of the dataset.

Logistic Regression is also a strong choice. It is computationally efficient and relatively simple to understand. If interpretability and computational efficiency are important, Logistic Regression may be preferred even if its accuracy is slightly lower than KNN.

Decision Tree has the advantage of interpretability. The rules learned by the tree can be visualized and explained easily. This can be useful in applications where understanding the reason behind a prediction is important.

Therefore, the final selection depends on the evaluation results and project requirements. For this particular experiment, KNN can be selected as the best-performing algorithm if it produces the highest accuracy and F1-score in the actual experiment.

The selection can be justified using the following criteria:

1. Highest or near-highest accuracy.
2. Strong precision across all three classes.
3. Strong recall across all three classes.
4. Highest or near-highest F1-score.
5. Fewest classification errors in the confusion matrix.

If the actual Python results show that another algorithm has the highest performance, the best-performing algorithm section should be updated accordingly.

It is also important to recognize that the difference between models may be very small. For example, if Logistic Regression achieves 96.67% accuracy and KNN achieves 100%, the difference is only one correctly classified observation in a 30-observation test set.

For this reason, cross-validation could be used to determine whether the apparent performance difference is consistent across different subsets of the dataset.

The best-performing algorithm should therefore be identified based on the actual experimental results rather than assumptions about which algorithm is theoretically better.

For the example experiment in this report, KNN is selected as the best-performing algorithm because it provides the highest overall classification performance. The model successfully uses the similarity between flower measurements to distinguish the three Iris species.

10. CONCLUSION

This project presented a complete machine learning classification workflow using the Iris flower dataset. The main objective was to apply three different machine learning algorithms, compare their performance using suitable evaluation metrics, and identify the best-performing algorithm.

The Iris dataset contains 150 observations belonging to three flower species: Iris-setosa, Iris-versicolor, and Iris-virginica. Four numerical measurements—sepal length, sepal width, petal length, and petal width—were used as input features for classification.

The data preprocessing stage involved examining the dataset, checking for missing values and duplicates, separating the independent and dependent variables, encoding the target where required, and splitting the dataset into training and testing subsets. Feature scaling was applied where appropriate, particularly because KNN relies on distance calculations.

Exploratory Data Analysis provided useful insights into the dataset. The visualizations demonstrated that the three species have different feature distributions. In particular, petal length and petal width were found to be useful for distinguishing the species. The dataset was also found to be balanced, with 50 observations from each class.

Three machine learning algorithms were implemented: Logistic Regression, Decision Tree, and K-Nearest Neighbors. Each algorithm uses a different approach to classification. Logistic Regression estimates class probabilities, Decision Tree learns a series of decision rules, and KNN uses the similarity between observations.

The models were evaluated using accuracy, precision, recall, F1-score, and confusion matrices. These metrics provided both an overall and detailed view of model performance.

The experiment showed that all three algorithms can achieve very high classification performance on the Iris dataset. This is largely due to the relatively clear separation between the species in the feature space.

Based on the example results, KNN achieved the highest overall performance and was selected as the best-performing algorithm. However, the actual best model should be determined from the results generated by the final Python implementation.

This project demonstrates that selecting a machine learning algorithm involves more than simply training a model. Data preprocessing, exploratory analysis, model selection, evaluation, and comparison are all important components of a successful machine learning project.

The project also demonstrates the importance of using multiple evaluation metrics. Accuracy alone may not provide enough information about model behavior, while precision, recall, F1-score, and confusion matrices provide additional insights.

As future work, the project could be extended by using cross-validation, hyperparameter tuning, additional classification algorithms such as Support Vector Machines and Random Forest, and larger real-world datasets.

Overall, the experiment successfully demonstrates how supervised machine learning can be used to classify Iris flower species and how different algorithms can be systematically compared to select an appropriate model.

11. REFERENCES

The following references were used as conceptual and technical resources for understanding the dataset, machine learning algorithms, preprocessing techniques, and evaluation metrics.

Books

1. Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. O'Reilly Media.
2. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). An Introduction to Statistical Learning: with Applications in R. Springer.
3. Han, J., Kamber, M., & Pei, J. (2011). Data Mining: Concepts and Techniques. Morgan Kaufmann.

Machine Learning Documentation

4. Scikit-learn Documentation. Machine Learning in Python. Scikit-learn provides implementations of Logistic Regression, Decision Tree, KNN, preprocessing methods, and evaluation metrics.
5. Scikit-learn documentation for LogisticRegression, used for implementing the Logistic Regression classification model.
6. Scikit-learn documentation for DecisionTreeClassifier, used for implementing the Decision Tree classification model.
7. Scikit-learn documentation for KNeighborsClassifier, used for implementing the KNearest Neighbors model. Dataset Reference
8. Fisher, R. A. (1936). The Use of Multiple Measurements in Taxonomic Problems. Annals of Eugenics, 7(2), 179–188.
9. The Iris dataset, originally introduced by Ronald A. Fisher, is widely used as a benchmark dataset for classification and statistical learning.